

SYSTEM ANALYST

presented by Ayub

UML: Activity Diagram

Вы уже умеете рисовать бизнес-процессы в BPMN.

Как думаете — зачем ещё одна похожая диаграмма в UML?

Activity Diagram — это BPMN для разработчиков

Те же идеи: поток действий, решения, параллельность.

Разница — не в смысле, а в аудитории и акцентах:

- BPMN → бизнес-аналитики, менеджеры, заказчики
- Activity Diagram → разработчики, UML-документация

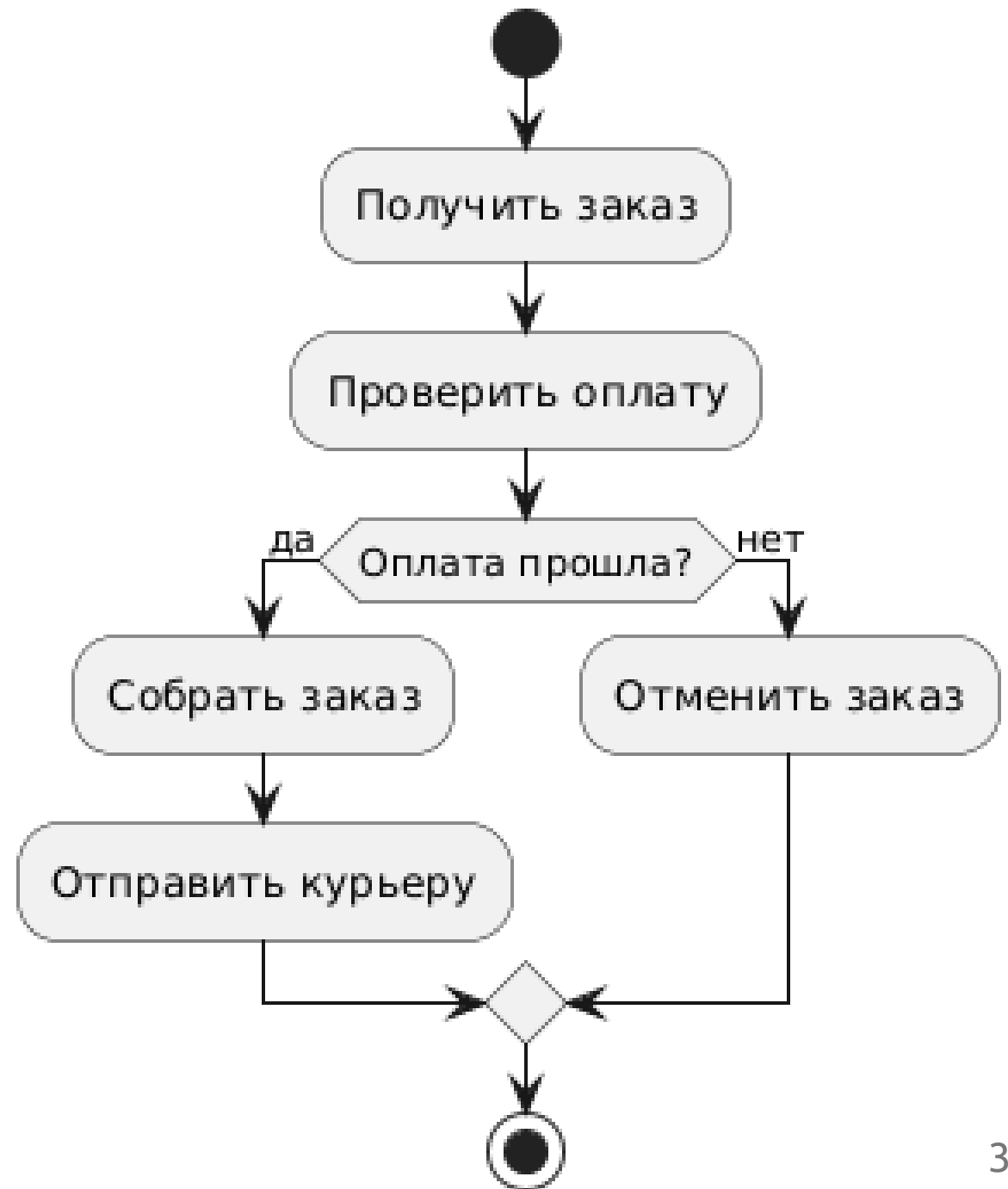
*Activity Diagram появилась **раньше BPMN** и стала частью UML 2.0.*

Когда проект уже использует UML — Activity Diagram естественный выбор.

Ты уже знаешь 90%

Посмотри на диаграмму справа. Что узнаёшь?

- ● закрашенный круг — начало
- Прямоугольники — действия
- Ромб — решение
- Стрелки — переходы
- ⊙ круг в круге — конец



Элементы Activity Diagram

Элемент	Символ	Аналог в BPMN
Initial Node	● (закрашенный круг)	Start Event (зелёный)
Action	▭ (закруглённый прямоугольник)	Task
Decision	◇ (ромб)	XOR Gateway
Fork / Join	—— (жирная полоска)	AND Gateway
Final Node	⊙ (круг в круге)	End Event (красный)
Swimlane	вертикальная полоса	Lane в Pool

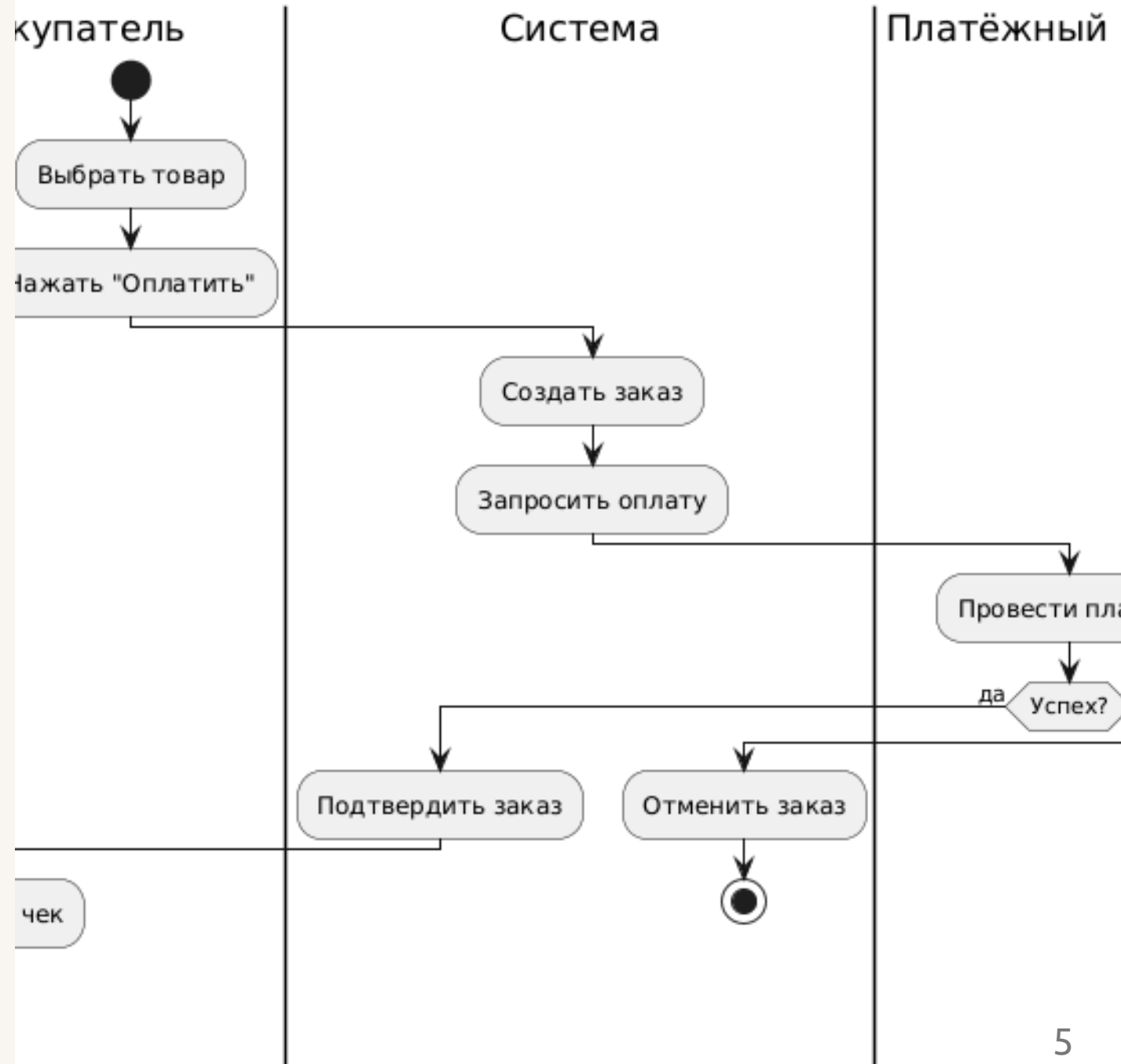
Знаешь BPMN — таблица читается за 30 секунд.

Swimlanes — кто что делает

Swimlane в Activity = Lane в BPMN:

- Разделяет, кто выполняет действие
- Полосы вертикальные или горизонтальные
- Пересечение линии = передача ответственности

Термин тот же — **Swimlane** — в обеих нотациях.



Когда ты выберешь BPMN, а когда Activity Diagram?

Два сценария:

1. Показать бизнесу, как работает процесс согласования заявки
2. Показать разработчику алгоритм обработки заказа в коде

Когда что использовать

Ситуация	Инструмент
Процесс для заказчика / менеджера	BPMN
Таймеры, сообщения, компенсации	BPMN
Алгоритм в системе для разработчиков	Activity
Описание поведения Use Case	Activity
Проект уже использует UML-нотацию	Activity

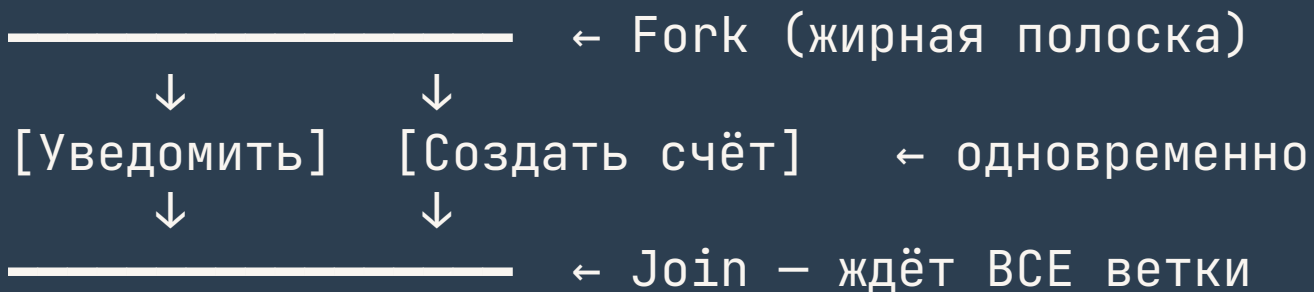
*Правило: **кто читает** — тот определяет нотацию.*

BPMN vs Activity Diagram

	BPMN	Activity Diagram
Аудитория	Бизнес + разработчики	Разработчики
Фокус	Бизнес-процесс	Алгоритм / поведение
События	Богатые (Timer, Message, Signal...)	Минимальные
Параллельность	AND Gateway	Fork / Join
Межпроцессное взаимодействие	Message Flow между Pools	Нет
Стандарт	OMG BPMN 2.0	UML 2.x

Fork и Join — параллельность

Fork — разделение на параллельные потоки:



- **Fork** = AND Gateway (split) в BPMN
- **Join** = AND Gateway (merge) в BPMN
- Join ждёт **все** параллельные ветки перед продолжением

PlantUML — Activity за 2 минуты

```
@startuml
start
:Получить заявку;
if (Заявка полная?) then (Да)
    fork
        :Уведомить менеджера;
    fork again
        :Создать задачу в Jira;
    end fork
    :Обработать заявку;
else (Нет)
    :Запросить доп. данные;
endif
stop
@enduml
```

start · :Action; · if/else/endif · fork/fork again/end fork

Инструменты

Инструмент	Как использовать
draw.io	Shapes → UML → Activity
PlantUML	Текстовый синтаксис, встраивается в Confluence/GitLab
Lucidchart	Визуальный редактор, шаблоны UML
StarUML	Десктоп, полный UML-набор

draw.io + PlantUML — самая частая комбинация в реальных SA-проектах.

Module Check

1. Назови **3 отличия** Activity Diagram от BPMN
2. Когда ты как SA выберешь Activity вместо BPMN?
3. Что такое **Fork** в Activity Diagram?
4. Как называется аналог BPMN Lane в Activity Diagram?

Чек-лист: Activity Diagram

Знаешь BPMN — Activity знаешь на 90%. Остальное — детали нотации.

- [] **Initial Node** (●) — начало процесса
- [] **Action** — прямоугольник с закруглёнными углами
- [] **Decision** (◇) — ветвление, условие на стрелке
- [] **Fork / Join** (——) — параллельность
- [] **Swimlane** — кто выполняет действие
- [] **Final Node** (⊙) — конец процесса

Что дальше

Следующий блок: **Data Modeling**

- **ERD** (Entity-Relationship Diagram) — проектирование структуры базы данных
- **SQL** — пишем запросы к спроектированным таблицам
- **Нормализация** — убираем дублирование данных

*Всё что изучили в UML — поможет понять структуру данных:
умение читать потоки и условия напрямую переносится на понимание того, как
данные меняются в системе.*